Dokumentation einer Verwandtschafts-Pipeline mit Snakemake: von Ganzgenomdaten zu Mendel-Fehlern und KING

Karim Andjar

Justus-Liebig-Universität Gießen, Master Bioinformatik und Systembiologie

19. Oktober 2026

Zusammenfassung

Diese Dokumentation beschreibt eine vollständige Pipeline, die aus Ganzgenomdaten von vier Personen der CEPH-Familie 1463 ableitet, wer mit wem verwandt ist, ohne dass Rollen vorgegeben werden. Die Verarbeitung läuft als Snakemake-Workflow auf einem Laptop: Herunterladen und Abgleich der Reads, Duplikatmarkierung, Variantenbestimmung im GVCF-Modus, gemeinsame Genotypisierung, Filterung auf verlässliche SNPs und zwei unabhängige Auswertungen, die Zählung von Mendel-Fehlern und der KING-robust-Schätzer. Für jeden Schritt werden die biologische Idee, das Werkzeug, die Snakemake-Regel, die Ein- und Ausgabedateien und die tatsächlich erzielten Zahlen erklärt. Ergänzt wird dies durch die Auswertungsskripte, die vier Abbildungen, eine Anleitung zur Reproduktion und eine Liste der aufgetretenen Probleme mit ihren Lösungen.

Abstract

This document describes a complete pipeline that infers relatedness among four members of the CEPH family 1463 from whole-genome data, without assigning roles in advance. A Snakemake workflow runs on a laptop and covers read download and alignment, duplicate marking, GVCF variant calling, joint genotyping, SNP filtering and two independent analyses, Mendelian error counting and the KING-robust estimator. For every step, the biological rationale, the tool, the Snakemake rule, the input and output files and the observed numbers are given, together with the analysis scripts, the four figures, reproduction instructions and a list of problems encountered and their solutions.

Schlüsselwörter: Verwandtschaftsanalyse, Mendel-Fehler, KING, Snakemake, GATK, CEPH 1463

1 Überblick

Ob zwei Personen verwandt sind, lässt sich aus ihren Genotypen ablesen. Ein Kind erbt von jedem Elternteil genau ein Allel. Elternteil und Kind können deshalb an keiner Position entgegengesetzte Homozygote tragen (AA gegen aa), während das bei nicht verwandten Personen regelmäßig vorkommt. Die Pipeline macht diese Überlegung mit echten Sequenzdaten überprüfbar und leitet die Rollen (Eltern, Kind, nicht verwandt) allein aus den Genotypen ab.

Der Workflow hat drei Teile (Abb. 1). Im ersten Teil werden aus den Reads je Person und Chromosom Varianten bestimmt. Im zweiten Teil werden alle Personen gemeinsam genotypisiert und auf verlässliche SNPs gefiltert, damit alle Vergleiche auf denselben Positionen beruhen. Im dritten Teil werten zwei unabhängige Verfahren die Verwandtschaft aus.

2 Umgebung und Installation

2.1 Rechner und Software

Die Pipeline läuft auf einem MacBook Air in einer conda-Umgebung mit dem Namen `trio_test`. Alle Programme stammen aus den Kanälen `bioconda` und `conda-forge` (Tab. 1). Der Workflow ist mit Snakemake (Mölder et al., 2021) umgesetzt.

Tabelle 1: Verwendete Software (getestete Versionen).

Programm	Version	Aufgabe
Snakemake	9.27.0	Workflow
bwa	0.7.19	Abgleich
samtools	1.24	Sortieren, Index
GATK	4.6.2.0	Duplikate, Varianten
bcftools	1.24	Filter, Zusammenfügen
PLINK 2	2.0.0a.7.8	KING
Python	3.11.17	Auswertung
Matplotlib	3.11.2	Abbildungen

Eine passende Umgebung lässt sich zum Beispiel so anlegen und aktivieren:

```
conda create -n trio_test -c conda-forge -c bioconda \
  snakemake bwa samtools gatk4 bcftools plink2 \
  matplotlib python=3.11
conda activate trio_test
```

2.2 Projektstruktur

```
trio_test/
|-- Snakefile        Workflow, alle Regeln
|-- config.yaml      Parameter und Proben
|-- scripts/
|   |-- mendel_check.py   Mendel-Fehler
|   `-- plot_results.py   vier Abbildungen
|-- data/            FASTQ (temporaer)
|-- work/            BAM, PLINK-Dateien
```

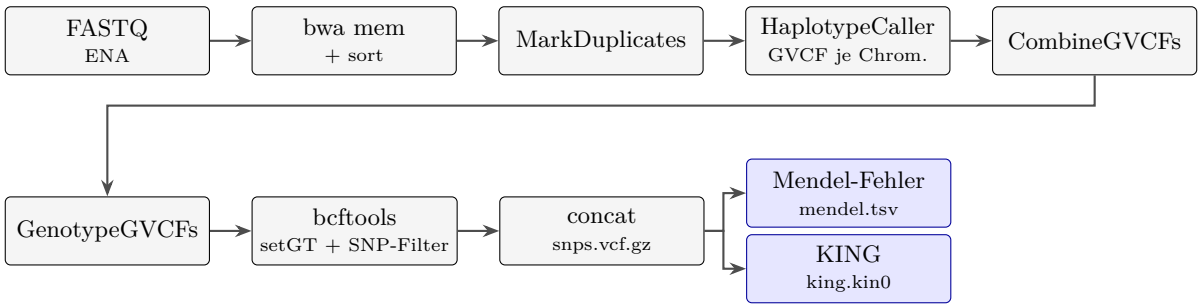


Abbildung 1: Aufbau der Pipeline. Teil 1 (obere Reihe): Reads bis GVCF. Teil 2 (untere Reihe links): gemeinsame Genotypisierung und Filter. Teil 3 (blau): Auswertung. Die Schritte FASTQ bis MarkDuplicates entfallen für Proben, für die eine fertige BAM vorliegt.

```
|-- qc/              Duplikat-Metriken
|-- gvcf/            GVCF je Probe, Chrom.
|-- joint/           gemeinsame Dateien
|-- filtered/        gefilterte SNPs
|-- results/         Endergebnisse
|   `-- abbildungen/ abb1 bis abb4
|-- bericht.tex, dokumentation.tex
`-- literatur.bib
```

Die Ordner `data/` und `work/` enthalten nur Dateien, die Snakemake nach Gebrauch löscht.

3 Daten und Referenz

3.1 Proben

Verwendet werden vier Personen der CEPH-Familie 1463 aus den Platinum Genomes (Eberle et al., 2017): NA12877, NA12878, NA12891 und NA12892. Für NA12877 und NA12878 lagen fertige Alignments (BAM) vor. Für NA12891 (Lauf ERR194160) und NA12892 (Lauf ERR194161) lädt die Pipeline die FASTQ-Dateien aus dem European Nucleotide Archive (ENA) und gleicht sie selbst ab. In `config.yaml` steht dafür je Probe entweder eine `accession` oder ein `bam`-Pfad:

```
samples:
  PROBE_A:
    accession: ERRxxxxxxx   # FASTQ von ENA
  PROBE_B:
    bam: /pfad/zu/probe_b.bam
```

Die mittlere Tiefe an den SNP-Stellen der gefilterten Genotypen beträgt 61,7 (NA12877), 58,8 (NA12878), 60,3 (NA12891) und 63,8 (NA12892).

3.2 Referenzgenom

Als Referenz dient GRCh38 in der Ensembl-Variante `Homo_sapiens.GRCh38.dna.primary_assembly.fa.gz`. Ihr Pfad steht in `config.yaml` unter `reference`. Für BWA, samtools und GATK müssen die zugehörigen Indexdateien vorhanden sein. Ensembl benennt die Chromosomen ohne Präfix (`20`, nicht `chr20`); im Workflow werden sie genauso angegeben.

3.3 Chromosomenauswahl

Ausgewertet werden die Chromosomen 20, 21 und 22 (`chroms` in der Konfiguration). Das hält den Rechenaufwand auf einem Laptop überschaubar. Bleibt die Liste leer, laufen die Chromosomen 1 bis 22.

4 Die Pipeline Schritt für Schritt

Alle Regeln stehen im `Snakefile` (Anhang A). Die Platzhalter `{sample}` und `{chrom}` stehen für Proben und Chromosomen aus der Konfiguration. Tabelle 2 fasst alle Regeln zusammen.

4.1 Reads herunterladen (`download_fastq`)

Idee. Ein Sequenzierer liest Fragmente der genomischen DNA von beiden Enden. Ein Fragment ergibt daher zwei zusammengehörige Reads (`_1` und `_2`, hier 101 bp).

Umsetzung. Die Regel lädt beide Dateien einer Probe über HTTPS aus dem ENA. `curl` setzt abgebrochene Downloads fort (`-C -`) und versucht es bis zu 50-mal mit 30 s Pause. Die fertige Datei wird mit `gzip -t` auf Vollständigkeit geprüft. Bis dahin liegt sie als `.part`-Datei in `data/`.

4.2 Abgleich (`align`)

Idee. Jeder Read muss seiner Herkunftsposition im Genom zugeordnet werden. BWA-MEM (Li, 2013) sucht dazu kurze exakte Übereinstimmungen mit der Referenz und verlängert sie zu einem vollständigen Alignment. Die *Read Group* (Felder `ID`, `SM`, `PL`, `LB`) ordnet die Reads einer Person zu. Das Feld `SM` ist das Probenlabel, das GATK später im VCF verwendet. Danach wird nach Position sortiert, weil Indexierung und Variantenbestimmung sortierte Dateien brauchen (Li et al., 2009).

```
bwa mem -t {threads} -R '{rg}' {ref} r1 r2 \
  | samtools sort -@ 4 -m 2G -o out.bam -
```

4.3 Duplikate markieren (`markdup`)

Idee. Bei der Bibliotheksherstellung wird DNA per PCR vervielfältigt. Mehrere Reads können dann Kopien desselben Fragments sein. Sie würden ein Allel

Tabelle 2: Regeln des Workflows mit Werkzeug, Eingabe und Ausgabe. Dateien mit * sind temporär und werden gelöscht.

Regel	Werkzeug	Eingabe	Ausgabe
download_fastq	curl	ENA-Accession	`data/{sample}_{1,2}.fastq.gz`*
align	bwa mem, samtools sort	FASTQ-Paar	`work/{sample}.sorted.bam`*
markdup	GATK MarkDuplicates	sortierte BAM	`work/{sample}.md.bam`*, `qc/{sample}.dup_metrics.txt`
haplotypecaller	GATK HaplotypeCaller	BAM (Config oder `md.bam`)	`gvcf/{sample}.{chrom}.g.vcf.gz`
combine_gvcfs	GATK CombineGVCFs	GVCF aller Proben je Chrom.	`joint/{chrom}.g.vcf.gz`
genotype	GATK GenotypeGVCFs	`joint/{chrom}.g.vcf.gz`	`joint/{chrom}.vcf.gz`
filter_snps	bcftools +setGT, view	`joint/{chrom}.vcf.gz`	`filtered/{chrom}.snps.vcf.gz`
concat	bcftools concat	gefilterte SNPs aller Chrom.	`results/snps.vcf.gz`
king	PLINK 2	`results/snps.vcf.gz`	`results/king.kin0`
mendel	Python	`results/snps.vcf.gz`	`results/mendel.tsv`
all	—	—	verlangt `king.kin0`, `mendel.tsv`

mehrfach zählen und eine Sicherheit vortäuschen, die es nicht gibt. GATK `MarkDuplicates` erkennt solche Reads an gleicher Start- und Endposition und markiert sie, ohne sie zu löschen. Der Variantenaufruf ignoriert markierte Reads. Die Metriken landen in `qc/{sample}.dup_metrics.txt`.

4.4 Varianten bestimmen (`haplotypecaller`)

Idee. Der *HaplotypeCaller* (McKenna et al., 2010) sucht Regionen, in denen die Reads von der Referenz abweichen, und setzt dort die möglichen Haplotypen lokal neu zusammen. Aus den Reads berechnet er für jede Position die Wahrscheinlichkeit der möglichen Genotypen (0/0, 0/1, 1/1).

GVCF-Modus. Mit `-ERC GVCF` schreibt GATK nicht nur Varianten, sondern auch Blöcke ohne Variante mit einer Aussage über die Sicherheit. So lässt sich später unterscheiden, ob eine Person an einer Stelle homozygot Referenz ist oder dort schlicht keine Daten hat. Die Bestimmung läuft je Probe und Chromosom (`-L {chrom}`).

Eingabe ist die fertige BAM aus der Konfiguration, sonst `work/{sample}.md.bam`.

4.5 Gemeinsam genotypisieren (`combine_gvcfs`, `genotype`)

Idee. Verwandtschaft vergleicht Personen Position für Position. Dafür müssen alle Personen an denselben Positionen einen Genotyp haben. `CombineGVCFs` fügt die GVCF-Dateien aller Proben eines Chromosoms zusammen, `GenotypeGVCFs` bestimmt daraus für jede variable Position den Genotyp aller Personen gemeinsam. Eine Variante, die nur bei einer Person gefunden wurde, bekommt so auch für die anderen einen Genotyp, entweder Referenz oder fehlend.

4.6 Auf verlässliche SNPs filtern (`filter_snps`)

Idee. Falsche Genotypen erzeugen falsche Mendel-Fehler, zum Beispiel ein heterozygoter Genotyp, der bei geringer Tiefe als homozygot erscheint. Der Filter schränkt deshalb auf sichere Genotypen ein:

```
bcftools +setGT in.vcf.gz -- -t q -n . \
  -i 'FMT/DP<15 || FMT/GQ<30' \
| bcftools view -m2 -M2 -v snps \
  -i 'QUAL>30 && F_MISSING=0' -Oz -o out.vcf.gz
```

Der erste Befehl setzt jeden Genotyp mit Tiefe unter `min_dp` (15) oder Genotypqualität unter `min_gq` (30) auf fehlend. Der zweite behält nur biallelische SNPs (`-m2 -M2 -v snps`) mit `QUAL>30`, bei denen *alle* Proben einen Genotyp haben (`F_MISSING=0`). Dadurch beruhen alle Paar- und Trio-Vergleiche auf denselben Positionen. Die Schwellen sind nicht an einem Referenzdatensatz kalibriert (Danecek et al., 2021).

4.7 Chromosomen zusammenfügen (`concat`)

`bcftools concat` fügt die gefilterten Chromosomen zu `results/snps.vcf.gz` zusammen und erzeugt den Index. Ergebnis: 329.422 SNPs auf den Chromosomen 20 bis 22.

4.8 Mendel-Fehler (`mendel`)

Idee. Das Skript `scripts/mendel_check.py` kennt keine Rollen und prüft alle Kombinationen. Die Genotypen werden als Dosis kodiert (0, 1 oder 2 Alternativallele). Haploide oder fehlende Genotypen werden übersprungen.

- **Paare.** Fehler ist ein SNP mit entgegengesetzten Homozygoten (Dosis 0 gegen 2).
- **Trios.** Fehler ist ein SNP, bei dem die Dosis des Kindes keine Summe aus je einem Allel der beiden

Eltern sein kann. Bei vier Personen gibt es zwölf Trios.

Die Fehlerrate ist der Anteil der Fehler an allen verglichenen SNPs. Der Aufruf lautet:

```
python scripts/mendel_check.py \
  --vcf results/snps.vcf.gz > results/mendel.tsv
```

4.9 KING (`king`)

Idee. KING-robust (Manichaikul et al., 2010) schätzt den Verwandtschaftskoeffizienten aus der Zahl gemeinsamer Allele, ohne dass dafür Allelfrequenzen einer Population nötig sind. Für Eltern und Kind wird ein Wert von etwa 0,25 erwartet, für nicht verwandte Personen 0. Zusätzlich gibt `IBS0` den Anteil der Positionen mit entgegengesetzten Homozygoten an.

Umsetzung. PLINK 2 (Chang et al., 2015) wandelt zuerst die VCF-Datei in das Binärformat um und vergibt Variantennamen aus Position und Allelen. Danach berechnet `--make-king-table` die Werte für alle Paare:

```
plink2 --vcf in.vcf.gz --double-id \
  --vcf-half-call m \
  --set-all-var-ids '@:#:$r:$a' \
  --new-id-max-allele-len 50 \
  --make-bed --out work/cohort
plink2 --bfile work/cohort --mac 1 \
  --make-king-table --out results/king
```

`--vcf-half-call m` behandelt halbe Aufrufe als fehlend, `--mac 1` entfernt Positionen ohne Variation. Dabei fallen 23.878 SNPs weg, es bleiben 305.544.

5 Ergebnisse und Rollenableitung

Die Rollen werden vom Plot-Skript aus den Daten abgeleitet. Bei den Paaren trennt die größte Lücke zwischen den sortierten Fehlerraten (logarithmisch) verwandte von nicht verwandten Paaren. Das Trio mit der kleinsten Fehlerrate legt Kind und Eltern fest.

Zwei Paare haben eine sehr niedrige Fehlerrate (Tab. 3): NA12878–NA12891 (0,015 %) und NA12878–NA12892 (0,096 %). Alle anderen Paare liegen zwischen 5,4 und 5,9 %, der Abstand beträgt etwa das 56-Fache. Unter den zwölf Trios hat nur NA12891 + NA12892 → NA12878 eine niedrige Rate (0,654 %), die nächstbeste Kombination liegt bei 14,1 %. KING bestätigt das mit IBS0 von 0,00016 und 0,0010 für die beiden Eltern-Kind-Paare gegenüber 0,058 bis 0,064 bei den übrigen. Das Ergebnis lautet: NA12878 ist das Kind von NA12891 und NA12892, NA12877 ist mit allen dreien nicht verwandt.

Die absoluten KINSHIP-Werte liegen für alle Paare über dem theoretischen Wert (etwa 0,31 für Eltern–Kind, 0,12 bis 0,14 für nicht verwandte Paare). Ich vermute, dass die kleine Kohorte die Schätzung verzerrt, habe das aber nicht überprüft. Belastbar sind IBS0 und die Rangfolge.

Tabelle 3: Mendel-Fehler pro Paar (329.422 SNPs).

Paar	Fehler	Rate
NA12878–NA12891	49	0,015 %
NA12878–NA12892	317	0,096 %
NA12877–NA12892	17.796	5,402 %
NA12877–NA12878	18.692	5,674 %
NA12877–NA12891	18.756	5,694 %
NA12891–NA12892	19.464	5,909 %

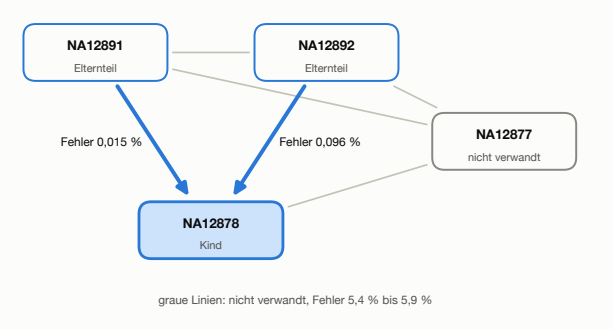


Abbildung 2: Abgeleitete Verwandtschaft. Die Pfeile gehen von den Eltern zum Kind, die Zahlen sind die Mendel-Fehlerraten.

6 Visualisierung

Die Abbildungen entstehen mit `scripts/plot_results.py` und Matplotlib (Hunter, 2007). Das Skript liest `results/mendel.tsv` und `results/king.kin0` und schreibt vier Abbildungen als PNG (300 dpi) und PDF nach `results/abbildungen/`. Blau steht durchgehend für Eltern–Kind, Grau für nicht verwandt. Die Fehlerraten stehen auf einer logarithmischen Achse, weil sie sich um mehr als zwei Größenordnungen unterscheiden. Abbildung 2 zeigt die abgeleitete Verwandtschaft, Abbildung 3 und 4 die Mendel-Fehlerraten der Paare und Trios und Abbildung 5 die KING-Werte.

7 Ausführung und Reproduktion

Alle Befehle laufen im Projektordner mit aktivierter Umgebung.

```
mkdir -p data work
snakemake -n
caffeinate -i snakemake -j 8 \
  --resources mem_gb=16 \
  --rerun-triggers mtime
python scripts/plot_results.py
```

`snakemake -n` ist ein Trockenlauf und zeigt, was laufen würde. `-j 8` erlaubt 8 gleichzeitige Kerne, `--resources mem_gb=16` begrenzt den verplanten Speicher (die Regeln melden je 8 GB an), und `--rerun-triggers mtime` startet nur Regeln neu, deren Eingabe neuer ist als die Ausgabe. `caffeinate -i` hält den Mac wach. Ein einzelnes Ziel wird mit dem Dateinamen vor den Optionen gestartet, zum Beispiel `snakemake results/snps.vcf.gz`

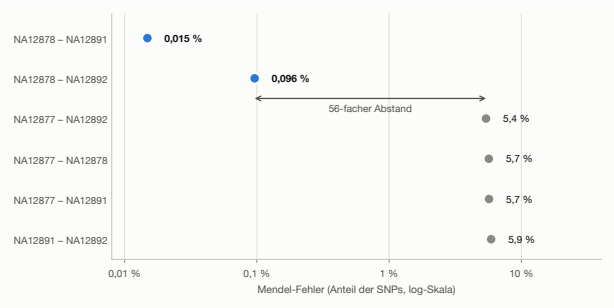


Abbildung 3: Mendel-Fehlerrate pro Paar (logarithmische Skala).

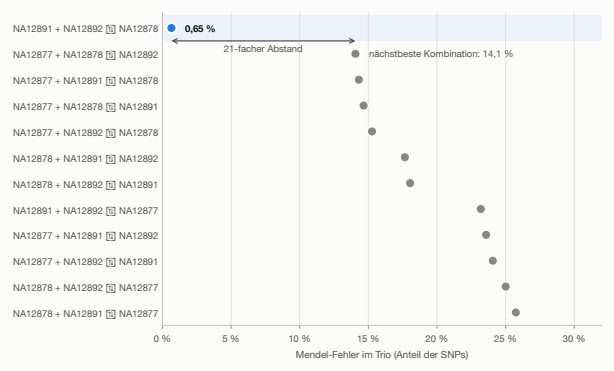


Abbildung 4: Mendel-Fehlerrate aller zwölf Trio-Kombinationen. Das beste Trio ist blau hervorgehoben.

`-rerun-triggers mtime`.

Der Bericht und diese Dokumentation werden mit LaTeX übersetzt (die Dokumentation braucht BibTeX für die Literatur):

```
pdflatex dokumentation.tex
bibtex dokumentation
pdflatex dokumentation.tex
pdflatex dokumentation.tex
```

8 Fehlersuche

`Failed to open work/cohort.log`. Die Regel `king` schreibt nach `work/`, aber Snakemake entfernt den Ordner, wenn nach dem Löschen der temporären BAM-Dateien nichts mehr darin liegt. Abhilfe: `mkdir -p work` und den Lauf wiederholen. Dauerhaft hilft ein `mkdir -p work` in der Regel `king` vor dem ersten `plink2`-Aufruf.

Snakemake will alles neu rechnen. Nach Änderungen am `Snakefile` erkennt Snakemake die Regeln als geändert. Mit `-rerun-triggers mtime` werden nur Regeln wiederholt, deren Eingabe neuer ist als die Ausgabe. Vorher mit `snakemake -n` prüfen.

Das Ziel wird als Ressource gelesen. `-resources` nimmt mehrere Werte entgegen. Ein Ziel nach dieser Option wird deshalb falsch gelesen. Das Ziel gehört vor die Optionen.

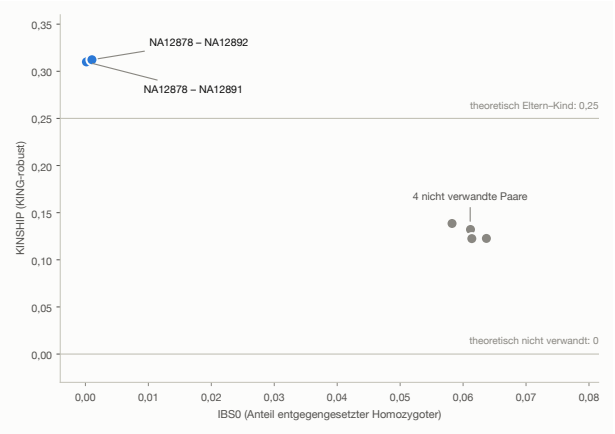


Abbildung 5: KING: IBS0 gegen KINSHIP. Die waagerechten Linien zeigen die theoretischen Erwartungswerte (0,25 und 0).

Download bricht ab. Kurze Verbindungs- und DNS-Ausfälle (curl-Fehler 18 und 6) fängt die Regel `download_fastq` selbst ab. Sie setzt fort und gibt nach 50 Versuchen auf. Bleibt der Fehler, später erneut starten.

Der Mac geht in den Ruhezustand. Den Lauf mit `caffeinate -i` starten.

Im PDF erscheinen `(?)` statt Literatur. Der BibTeX-Schritt ist nicht gelaufen. Die Kette `pdflatex`, `bibtex`, zweimal `pdflatex` ausführen und prüfen, dass `literatur.bib` im selben Ordner liegt.

9 Grenzen

- Es wurden nur drei Chromosomen und vier Personen untersucht. Die Aussage gilt für diese Daten.
- Vater und Mutter lassen sich mit autosomalen Daten nicht unterscheiden. Dafür wäre Chromosom X nötig. Das Mendel-Skript überspringt haploide Aufrufe und müsste dafür angepasst werden.
- Die Genotypfilter (`min_dp`, `min_gq`) sind nicht kalibriert, und eine Rekalibrierung der Basenqualitäten (BQSR) ist nicht Teil der Pipeline.
- Die Ursache der verschobenen KINSHIP-Werte ist nicht geklärt.
- Die Abbildungsskripte sind nicht in Snakemake eingebunden und laufen von Hand.

Literatur

Christopher C. Chang, Carson C. Chow, Laurent C. A. M. Tellier, Shashaank Vattikuti, Shaun M. Purcell, and James J. Lee. Second-generation PLINK: rising to the challenge of larger and richer datasets. *GigaScience*, 4: 7, 2015.

Petr Danecek, James K. Bonfield, Jennifer Liddle, et al. Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2):giab008, 2021.

Michael A. Eberle, Epameinondas Fritzilas, Peter Krusche, et al. A reference data set of 5.4 million phased human variants validated by genetic inheritance from sequencing a three-generation 17-member pedigree. *Genome Research*, 27:157–164, 2017.

John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

Heng Li. Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM, 2013.

Heng Li, Bob Handsaker, Alec Wysoker, et al. The sequence alignment/map format and SAMtools. *Bioinformatics*, 25:2078–2079, 2009.

Ani Manichaikul, Josyf C. Mychaleckyj, Stephen S. Rich, Kathy Daly, Michèle Sale, and Wei-Min Chen. Robust relationship inference in genome-wide association studies. *Bioinformatics*, 26:2867–2873, 2010.

Aaron McKenna, Matthew Hanna, Eric Banks, et al. The genome analysis toolkit: a MapReduce framework for analyzing next-generation DNA sequencing data. *Genome Research*, 20:1297–1303, 2010.

Felix Mölder, Kim Philipp Jablonski, Brice Letcher, et al. Sustainable data analysis with Snakemake. *F1000Research*, 10:33, 2021.

A Skripte

Snakefile

```
configfile: "config.yaml"

SAMPLES = list(config["samples"])
CHROMS = [str(c) for c in config["chroms"]] or [str(i) for i
    ↪ in range(1, 23)]
REF = config["reference"]
ALIGNER = config.get("aligner", "bwa")


def bam_of(w):
    # fertige BAM aus der Config, sonst die selbst erzeugte
    return config["samples"][w.sample].get("bam") or f"work/{
      ↪ w.sample}.md.bam"


wildcard_constraints:
    sample="|".join(SAMPLES),
    chrom="|".join(CHROMS),


rule all:
    input: "results/king.kin0", "results/mendel.tsv"


def ena_url(acc):
    num = acc[3:]
    sub = {6: "", 7: f"/00{num[-1]}", 8: f"/0{num[-2:]}", 9:
      ↪  f"/{num[-3:]}"}[len(num)]
    return f"ftp://ftp.sra.ebi.ac.uk/vol1/fastq/{acc[:6]}{
      ↪ sub}/{acc}"


rule download_fastq:
    output:
        temp("data/{sample}_{read}.fastq.gz"),
    wildcard_constraints:
        read="1|2",
    params:
        url=lambda w: ena_url(config["samples"][w.sample]["
      ↪ accession"]).replace("ftp://", "https://"),
        acc=lambda w: config["samples"][w.sample]["accession
      ↪ "],
    shell:
        """
        n=0
        until curl -fL -C - --connect-timeout 60 --speed-
      ↪ limit 50000 --speed-time 120 -o {output}.part {
      ↪ params.url}/{params.acc}_{wildcards.read}.fastq.gz;
      ↪ do
            n=$((n+1))
            if [ $n -ge 50 ]; then echo "Download nach 50
      ↪ Versuchen aufgegeben" >&2; exit 1; fi
            echo "Versuch $n fehlgeschlagen, warte 30 s und
      ↪ setze fort ..." >&2
            sleep 30
        done
        gzip -t < {output}.part
        mv {output}.part {output}
        """


rule align:
    input:
        r1="data/{sample}_1.fastq.gz",
        r2="data/{sample}_2.fastq.gz",
    output: bam=temp("work/{sample}.sorted.bam")
    threads: config["threads"]
    resources: mem_gb=config.get("align_mem_gb", 8)
    params:
        ref=REF, aligner=ALIGNER,
        rg=lambda w: f"@RG\\tID:{w.sample}\\tSM:{w.sample}\\
      ↪ tPL:ILLUMINA\\tLB:{w.sample}",
    shell:
        "{params.aligner} mem -t {threads} -R '{params.rg}' {
      ↪ params.ref} {input.r1} {input.r2} "
        "| samtools sort -@ 4 -m 2G -o {output.bam} -"


rule markdup:
    input: "work/{sample}.sorted.bam"
    output:
        bam=temp("work/{sample}.md.bam"),
        bai=temp("work/{sample}.md.bam.bai"),
        metrics="qc/{sample}.dup_metrics.txt",
    resources: mem_gb=8
    shell:
        "gatk --java-options '-Xmx6g' MarkDuplicates -I {
      ↪ input} -O {output.bam} -M {output.metrics} "
        "&& samtools index {output.bam}"


rule haplotypecaller:
    input:
        bam=bam_of,
        bai=lambda w: bam_of(w) + ".bai",
    output: "gvcf/{sample}.{chrom}.g.vcf.gz"
    threads: 2
    resources: mem_gb=8
    params: ref=REF
    shell:
        "gatk --java-options '-Xmx6g' HaplotypeCaller -R {
      ↪ params.ref} -I {input.bam} "
        "-L {wildcards.chrom} -O {output} -ERC GVCF --native-
      ↪ pair-hmm-threads {threads}"


rule combine_gvcfs:
    input: expand("gvcf/{sample}.{{chrom}}.g.vcf.gz", sample
      ↪ =SAMPLES)
    output: "joint/{chrom}.g.vcf.gz"
    resources: mem_gb=8
    params:
        ref=REF,
        v=lambda w, input: " ".join(f"-V {f}" for f in input)
      ↪ ,
    shell:
        "gatk --java-options '-Xmx6g' CombineGVCFs -R {
      ↪ params.ref} {params.v} "
        "-L {wildcards.chrom} -O {output}"


rule genotype:
    input: "joint/{chrom}.g.vcf.gz"
    output: "joint/{chrom}.vcf.gz"
    resources: mem_gb=8
    params: ref=REF
    shell:
        "gatk --java-options '-Xmx6g' GenotypeGVCFs -R {
      ↪ params.ref} -V {input} "
        "-L {wildcards.chrom} -O {output}"


rule filter_snps:
    input: "joint/{chrom}.vcf.gz"
    output: "filtered/{chrom}.snps.vcf.gz"
```

```
    params: dp=config["min_dp"], gq=config["min_gq"]
    shell:
        "bcftools +setGT {input} -- -t q -n . -i 'FMT/DP<{
      ↪ params.dp} || FMT/GQ<{params.gq}' "
        "| bcftools view -m2 -M2 -v snps -i 'QUAL>30 &&
      ↪ F_MISSING=0' -Oz -o {output} "
        "&& bcftools index -t {output}"


rule concat:
    input: expand("filtered/{chrom}.snps.vcf.gz", chrom=
      ↪ CHROMS)
    output: "results/snps.vcf.gz"
    shell: "bcftools concat -Oz -o {output} {input} &&
      ↪ bcftools index -t {output}"


rule king:
    input: "results/snps.vcf.gz"
    output: "results/king.kin0"
    params: out="results/king"
    shell:
        """
        plink2 --vcf {input} --double-id --vcf-half-call m \
               --set-all-var-ids '@:#:$r:$a' --new-id-max-
      ↪ allele-len 50 \
               --make-bed --out work/cohort
        plink2 --bfile work/cohort --mac 1 --make-king-table
      ↪  --out {params.out}
        """


rule mendel:
    input: "results/snps.vcf.gz"
    output: "results/mendel.tsv"
    shell: "python scripts/mendel_check.py --vcf {input} > {
      ↪ output}"
```

mendel_check.py

```
#!/usr/bin/env python3
import argparse
import subprocess
from itertools import combinations

ALLELES = {0: (0,), 1: (0, 1), 2: (1,)}

def dosage(gt):
    p = gt.replace("|", "/").split("/")
    if len(p) != 2 or "." in p:
        return None
    return int(p[0]) + int(p[1])

def pair_error(a, b):
    return (a == 0 and b == 2) or (a == 2 and b == 0)

def trio_error(f, m, c):
    return c not in {x + y for x in ALLELES[f] for y in
      ↪ ALLELES[m]}

def count(rows, n):
    pairs = list(combinations(range(n), 2))
    trios = [(a, b, c) for c in range(n)
             for a, b in combinations([i for i in range(n) if
      ↪  i != c], 2)]
    sp = {k: [0, 0] for k in pairs}
    st = {k: [0, 0] for k in trios}
    for g in rows:
        for a, b in pairs:
            if g[a] is None or g[b] is None:
                continue
            sp[(a, b)][0] += 1
            sp[(a, b)][1] += pair_error(g[a], g[b])
        for a, b, c in trios:
            if g[a] is None or g[b] is None or g[c] is None:
                continue
            st[(a, b, c)][0] += 1
            st[(a, b, c)][1] += trio_error(g[a], g[b], g[c])
    return sp, st

def report(sp, st, names):
    print("## Paare (sortiert nach Fehlerrate)")
    print("probe1\tprobe2\tsites\tmendel_errors\terror_rate")
      ↪
    for (a, b), (n, e) in sorted(sp.items(), key=lambda kv:
      ↪ kv[1][1] / max(kv[1][0], 1)):
        print(f"{names[a]}\t{names[b]}\t{n}\t{e}\t{e / n if
      ↪ n else float('nan'):.5f}")
    print("\n## Trios (sortiert nach Fehlerrate)")
    print("elter1\telter2\tkind\tsites\tmendel_errors\
      ↪ terror_rate")
    for (a, b, c), (n, e) in sorted(st.items(), key=lambda kv
      ↪ : kv[1][1] / max(kv[1][0], 1)):
        print(f"{names[a]}\t{names[b]}\t{names[c]}\t{n}\t{e}\
      ↪ t{e / n if n else float('nan'):.5f}")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--vcf", required=True)
    a = ap.parse_args()
    names = subprocess.run(["bcftools", "query", "-l", a.vcf
      ↪ ],
                           capture_output=True, text=True,
      ↪ check=True).stdout.split()
    proc = subprocess.Popen(["bcftools", "query", "-f", "[%
      ↪ GT\t]\n", a.vcf],
                            stdout=subprocess.PIPE, text=
      ↪ True)
    rows = ([dosage(g) for g in l.rstrip("\n").rstrip("\t").
      ↪ split("\t")] for l in proc.stdout)
    sp, st = count(rows, len(names))
    report(sp, st, names)

if __name__ == "__main__":
    main()
```

plot_results.py

```
#!/usr/bin/env python3
"""Abbildung der Verwandtschaftsanalyse aus mendel.tsv und
      ↪ king.kin0.

Die Rollen (Eltern, Kind, nicht verwandt) werden NICHT
      ↪ vorgegeben, sondern aus den
Daten abgeleitet:
  - Paare: Trennung an der größten Lücke der Mendel-
      ↪ Fehlerraten (log-Skala)
  - Trio:  das Trio mit der kleinsten Fehlerrate liefert
      ↪ Kind und Eltern
"""
import argparse
import math
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.patches import FancyBboxPatch
from matplotlib.ticker import FixedLocator, FuncFormatter,
      ↪ NullLocator

#
      ↪ ------------------------------------------------------------
      ↪  Farben ----
SURFACE = "#fcfcfb"
INK, INK2, MUTED = "#0b0b0b", "#52514e", "#898781"
GRID, AXIS = "#e1e0d9", "#c3c2b7"
REL = "#2a78d6"      # Eltern-Kind (hervorgehoben)
UNREL = "#898781"    # nicht verwandt (zurückgenommen)
REL_TINT = "#cde2fb"

plt.rcParams.update({
    "font.family": "sans-serif",
    "font.sans-serif": ["Inter", "Helvetica Neue", "
      ↪ Helvetica", "Arial", "DejaVu Sans"],
    "figure.facecolor": SURFACE,
    "axes.facecolor": SURFACE,
    "savefig.facecolor": SURFACE,
    "text.color": INK,
    "axes.edgecolor": AXIS,
    "axes.labelcolor": INK2,
    "xtick.color": MUTED,
    "ytick.color": INK2,
    "xtick.labelcolor": INK2,
    "pdf.fonttype": 42,
})


# ----------------------------------------------------
      ↪ Zahlenformatierung ----
def de(x, nd=2):
    return f"{x:.{nd}f}".replace(".", ",")
```

```
def pct(frac):
    """Anteil -> '0,015 %' mit sinnvoller Stellenzahl (
     ↪ deutsches Komma)."""
    v = 100 * frac
    nd = 3 if v < 0.1 else 2 if v < 1 else 1
    return de(v, nd) + "\u00a0%"


def thousands(n):
    return f"{n:,}".replace(",", ".")


#
     ↪ ------------------------------------------------------------
     ↪  Einlesen ----
def read_mendel(path):
    pairs, trios, section, header = [], [], None, None
    for line in Path(path).read_text().splitlines():
        s = line.strip()
        if not s:
            continue
        if s.startswith("##"):
            section = "pairs" if "Paar" in s else "trios"
            header = None
            continue
        cols = s.split("\t")
        if header is None:
            header = cols
            continue
        r = dict(zip(header, cols))
        sites, err = int(r["sites"]), int(r["mendel_errors"])
     ↪
        if section == "pairs":
            pairs.append(dict(a=r["probe1"], b=r["probe2"],
     ↪ sites=sites, err=err, rate=err / sites))
        else:
            trios.append(dict(p1=r["elter1"], p2=r["elter2"],
     ↪  child=r["kind"],
                              sites=sites, err=err, rate=err
     ↪  / sites))
    return pairs, trios


def read_king(path):
    rows = [l.split() for l in Path(path).read_text().
     ↪ splitlines() if l.strip()]
    header = [h.lstrip("#") for h in rows[0]]
    out = []
    for r in rows[1:]:
        d = dict(zip(header, r))
        out.append(dict(a=d["IID1"], b=d["IID2"], nsnp=int(d
     ↪ ["NSNP"]),
                        ibs0=float(d["IBS0"]), kin=float(d["
     ↪ KINSHIP"])))
    return out


def pair_key(a, b):
    return frozenset((a, b))


def pair_label(a, b):
    return " - ".join(sorted((a, b)))


# ------------------------------------------------ Rollen aus
     ↪ den Daten ----
def split_pairs(pairs, min_ratio=5.0):
    """Schwelle an der größten Lücke (log) zwischen den
     ↪ sortierten Fehlerraten."""
    rates = sorted(max(p["rate"], 1e-7) for p in pairs)
    gaps = [(math.log(rates[i + 1] / rates[i]), i) for i in
     ↪ range(len(rates) - 1)]
    g, i = max(gaps)
    if g < math.log(min_ratio):
        return None
    return math.sqrt(rates[i] * rates[i + 1])


# ------------------------------------------------------------
     ↪  Gestaltung ----
def style_axis(ax, grid="x"):
    for side in ("top", "right"):
        ax.spines[side].set_visible(False)
    for side in ("left", "bottom"):
        ax.spines[side].set_color(AXIS)
        ax.spines[side].set_linewidth(0.8)
    ax.tick_params(length=3, width=0.8, labelsize=8.5)
    ax.set_axisbelow(True)
    if grid:
        ax.grid(axis=grid, color=GRID, linewidth=0.7)


def panel_title(fig, ax, letter, text, x):
    """Titel als Figure-Text: x in Figure-Koordinaten, y
     ↪ knapp über dem Achsenkasten."""
    y = ax.get_position(original=True).y1 + 0.016
    fig.text(x, y, f"{letter}  {text}", fontsize=10.5,
     ↪ fontweight="bold",
             color=INK, ha="left", va="bottom")


def ring(color):
    return dict(s=78, color=color, edgecolor=SURFACE,
     ↪ linewidth=1.6, zorder=3)


# ------------------------------------------------------
     ↪ Panel A: Netz ----
def box_edge_point(c, target, w, h, pad=0.06):
    """Punkt, an dem die Linie von c nach target den Kasten
     ↪ (Breite w, Höhe h) verlässt."""
    dx, dy = target[0] - c[0], target[1] - c[1]
    if dx == 0 and dy == 0:
        return c
    tx = (w / 2 + pad) / abs(dx) if dx else float("inf")
    ty = (h / 2 + pad) / abs(dy) if dy else float("inf")
    t = min(tx, ty)
    return (c[0] + dx * t, c[1] + dy * t)


def draw_network(ax, names, pairs, trio, cut):
    W, H = 1.62, 0.64
    ax.set_xlim(-2.2, 4.1)
    ax.set_ylim(-1.95, 1.45)
    ax.set_aspect("equal", adjustable="box")
    ax.set_anchor("N")
    ax.axis("off")

    related = lambda p: cut is not None and p["rate"] < cut
    rate = {pair_key(p["a"], p["b"]): p["rate"] for p in
     ↪ pairs}

    # Positionen: Trio als Stammbaum-Dreieck, übrige Personen
     ↪  rechts
    if trio:
        pos = {trio["p1"]: (-1.3, 1.0), trio["p2"]: (1.3,
     ↪ 1.0), trio["child"]: (0.0, -1.0)}
        rest = [n for n in names if n not in pos]
        for i, n in enumerate(rest):
            pos[n] = (3.3, 0.0 if len(rest) == 1 else 1.0 -
     ↪ 2.0 * i / (len(rest) - 1))
        role = {trio["p1"]: "Elternteil", trio["p2"]: "
     ↪ Elternteil", trio["child"]: "Kind"}
        for n in rest:
            role[n] = "nicht verwandt"
    else:
        pos, role = {}, {n: "" for n in names}
        for i, n in enumerate(names):
            ang = 2 * math.pi * i / len(names)
            pos[n] = (1.8 * math.cos(ang), 1.2 * math.sin(
     ↪ ang))

    # 1) nicht verwandte Paare: dünne, zurückgenommene Linien
    for p in pairs:
        if related(p):
            continue
        a, b = pos[p["a"]], pos[p["b"]]
        pa, pb = box_edge_point(a, b, W, H), box_edge_point(
     ↪ b, a, W, H)
        ax.plot([pa[0], pb[0]], [pa[1], pb[1]], color=AXIS,
     ↪ lw=1.2, zorder=1,
                solid_capstyle="round")

    # 2) Eltern-Kind-Paare: dicke blaue Pfeile Richtung Kind
     ↪ (Richtung aus dem Trio)
    for p in pairs:
        if not related(p):
            continue
        a, b = p["a"], p["b"]
        if trio and b == trio["child"]:
            src, dst = a, b
        elif trio and a == trio["child"]:
            src, dst = b, a
        else:
            src, dst = a, b
        s, d = pos[src], pos[dst]
        ps, pd = box_edge_point(s, d, W, H), box_edge_point(
     ↪ d, s, W, H)
        ax.annotate("", xy=pd, xytext=ps, zorder=2,
                    arrowprops=dict(arrowstyle="-|>,
     ↪ head_length=0.7,head_width=0.32",
```

```
                                color=REL, lw=2.8,
    ↪ shrinkA=0, shrinkB=0))
        mid = ((ps[0] + pd[0]) / 2, (ps[1] + pd[1]) / 2)
        e = next(q for q in pairs if pair_key(q["a"], q["b"])
    ↪  == pair_key(a, b))
        side_left = (s[0] < d[0]) if s[0] != d[0] else True
        ax.text(mid[0] + (-0.12 if side_left else 0.12), mid
    ↪ [1], f"Fehler {pct(e['rate'])}",
                ha="right" if side_left else "left", va="
    ↪ center", fontsize=8.5,
                color=INK, zorder=4,
                bbox=dict(boxstyle="round,pad=0.18", fc=
    ↪ SURFACE, ec="none"))

    # 3) Knoten
    for n in names:
        x, y = pos[n]
        is_child = trio and n == trio["child"]
        is_parent = trio and n in (trio["p1"], trio["p2"])
        ec = REL if (is_child or is_parent) else UNREL
        fc = REL_TINT if is_child else SURFACE
        ax.add_patch(FancyBboxPatch((x - W / 2, y - H / 2),
    ↪ W, H,
                                    boxstyle="round,pad=0,
    ↪ rounding_size=0.14",
                                    fc=fc, ec=ec, lw=2.0 if
    ↪ is_child else 1.4, zorder=5))
        ax.text(x, y + 0.08, n, ha="center", va="center",
    ↪ fontsize=9.5,
                fontweight="bold", color=INK, zorder=6)
        ax.text(x, y - 0.17, role[n], ha="center", va="
    ↪ center", fontsize=7.8,
                color=INK2, zorder=6)

    # 4) Fehlerrate der nicht verwandten Paare als eine Zeile
    ↪  je Gruppe
    unrel_rates = [p["rate"] for p in pairs if not related(p)
    ↪ ]
    if unrel_rates:
        lo, hi = min(unrel_rates), max(unrel_rates)
        txt = f"graue Linien: nicht verwandt, Fehler {pct(lo)
    ↪ } bis {pct(hi)}"
        ax.text(0.5 * (ax.get_xlim()[0] + ax.get_xlim()[1]) +
    ↪  0.0, -1.78, txt,
                ha="center", va="center", fontsize=8.3,
    ↪ color=INK2)


# ------------------------------------------------ Panel B:
    ↪ Paare (log) ----
def draw_pairs(ax, pairs, cut):
    rows = sorted(pairs, key=lambda p: p["rate"])
    n = len(rows)
    ys = list(range(n))
    for y, p in zip(ys, rows):
        rel = cut is not None and p["rate"] < cut
        ax.scatter(100 * p["rate"], y, **ring(REL if rel
    ↪ else UNREL))
        ax.text(100 * p["rate"] * 1.35, y, pct(p["rate"]),
    ↪ va="center", ha="left",
                fontsize=8.5, color=INK, fontweight="bold"
    ↪ if rel else "normal")
    ax.set_yticks(ys)
    ax.set_yticklabels([pair_label(p["a"], p["b"]) for p in
    ↪ rows], fontsize=8.8)
    ax.invert_yaxis()
    ax.set_xscale("log")
    ax.set_xlim(0.008, 40)
    ax.xaxis.set_major_locator(FixedLocator([0.01, 0.1, 1,
    ↪ 10]))
    ax.xaxis.set_minor_locator(NullLocator())
    ax.xaxis.set_major_formatter(FuncFormatter(lambda v, _:
    ↪ de(v, 2 if v < 0.1 else 1 if v < 1 else 0) + "\u00a0
    ↪ %"))
    ax.set_xlabel("Mendel-Fehler (Anteil der SNPs, log-Skala)
    ↪ ", fontsize=8.8)
    ax.set_ylim(n - 0.5, -0.6)
    style_axis(ax, grid="x")
    ax.tick_params(axis="y", length=0)

    if cut is not None:
        k = sum(1 for p in rows if p["rate"] < cut)
        if 0 < k < n:
            a, b = 100 * rows[k - 1]["rate"], 100 * rows[k][
    ↪ "rate"]
            ymid = k - 0.5
            ax.annotate("", xy=(b, ymid), xytext=(a, ymid),
                        arrowprops=dict(arrowstyle="<->",
    ↪ color=INK2, lw=1.1,
                                        shrinkA=0, shrinkB=0)
    ↪ )
            ax.text(math.sqrt(a * b), ymid + 0.1, f"{math.
    ↪ floor(b / a)}-facher Abstand",
                    ha="center", va="top", fontsize=8.5,
    ↪ color=INK2)


# ------------------------------------------------ Panel C:
    ↪ Trios ----
def draw_trios(ax, trios):
    rows = sorted(trios, key=lambda t: t["rate"])
    n = len(rows)
    best = rows[0]
    for y, t in enumerate(rows):
        is_best = y == 0
        ax.scatter(100 * t["rate"], y, **ring(REL if is_best
    ↪  else UNREL))
    ax.axhspan(-0.5, 0.5, color=REL_TINT, alpha=0.35, lw=0,
    ↪ zorder=0)
    ax.set_yticks(range(n))
    ax.set_yticklabels([f"{t['p1']} + {t['p2']} → {t['child
    ↪ ']}" for t in rows], fontsize=8.2)
    ax.invert_yaxis()
    ax.set_ylim(n - 0.5, -0.6)
    xmax = 100 * max(t["rate"] for t in rows)
    ax.set_xlim(0, math.ceil(xmax / 5.0) * 5 + 2)
    ax.xaxis.set_major_formatter(FuncFormatter(lambda v, _:
    ↪ f"{int(v)}\u00a0%"))
    ax.set_xlabel("Mendel-Fehler im Trio (Anteil der SNPs)",
    ↪  fontsize=8.8)
    style_axis(ax, grid="x")
    ax.tick_params(axis="y", length=0)

    ax.text(100 * best["rate"] + 1.0, 0, pct(best["rate"]),
    ↪ va="center", ha="left",
            fontsize=8.5, fontweight="bold", color=INK)
    if n > 1:
        nxt = rows[1]
        ax.text(100 * nxt["rate"] + 1.0, 1, f"nächstbeste
    ↪ Kombination: {pct(nxt['rate'])}",
                va="center", ha="left", fontsize=8.5, color=
    ↪ INK2)
        ratio = nxt["rate"] / best["rate"]
        ax.annotate("", xy=(100 * nxt["rate"], 0.5), xytext
    ↪ =(100 * best["rate"], 0.5),
                    arrowprops=dict(arrowstyle="<->", color=
    ↪ INK2, lw=1.1, shrinkA=0, shrinkB=0))
        ax.text(50 * (best["rate"] + nxt["rate"]), 0.58, f"{
    ↪ math.floor(ratio)}-facher Abstand",
                ha="center", va="top", fontsize=8.5, color=
    ↪ INK2)


# ------------------------------------------------ Panel D:
    ↪ KING ----
def draw_king(ax, king, pairs, cut):
    rate = {pair_key(p["a"], p["b"]): p["rate"] for p in
    ↪ pairs}
    pts = []
    for k in king:
        r = rate.get(pair_key(k["a"], k["b"]))
        rel = cut is not None and r is not None and r < cut
        pts.append((k, rel))

    xmax = max(k["ibs0"] for k, _ in pts)
    ymax = max(k["kin"] for k, _ in pts)
    ax.set_xlim(-0.004, xmax * 1.28)
    ax.set_ylim(-0.03, max(0.36, ymax * 1.12))

    # theoretische Erwartungswerte als ruhige Referenzlinien
    for yv, txt in ((0.25, "theoretisch Eltern-Kind: 0,25"),
    ↪  (0.0, "theoretisch nicht verwandt: 0")):
        ax.axhline(yv, color=AXIS, lw=0.9, zorder=1)
        ax.text(ax.get_xlim()[1], yv + 0.008, txt, ha="right
    ↪ ", va="bottom", fontsize=8,
                color=MUTED)

    for k, rel in pts:
        ax.scatter(k["ibs0"], k["kin"], **ring(REL if rel
    ↪ else UNREL))

    # direkte Beschriftung mit Leitlinie: Eltern-Kind-Punkte
    ↪ liegen dicht beieinander
    rel_pts = sorted([k for k, rel in pts if rel], key=lambda
    ↪  k: k["ibs0"])
    offsets = [(46, -22), (46, 14)]          # links
    ↪ liegender Punkt -> Label unten, rechter -> oben
    for k, off in zip(rel_pts, offsets):
        ax.annotate(pair_label(k["a"], k["b"]), (k["ibs0"],
    ↪ k["kin"]),
                    xytext=off, textcoords="offset points",
    ↪ fontsize=8.5, color=INK,
```

```
                    ha="left", va="center",
                    arrowprops=dict(arrowstyle="-", color=
    ↪ MUTED, lw=0.9, shrinkA=2, shrinkB=5))
    unrel_pts = [k for k, rel in pts if not rel]
    if unrel_pts:
        cx = sum(k["ibs0"] for k in unrel_pts) / len(
    ↪ unrel_pts)
        cy = sum(k["kin"] for k in unrel_pts) / len(unrel_pts
    ↪ )
        ax.annotate(f"{len(unrel_pts)} nicht verwandte Paare
    ↪ ", (cx, cy),
                    xytext=(0, 24), textcoords="offset
    ↪ points", fontsize=8.5, color=INK2,
                    ha="center", va="bottom",
                    arrowprops=dict(arrowstyle="-", color=
    ↪ MUTED, lw=0.9, shrinkA=2, shrinkB=9))

    ax.xaxis.set_major_formatter(FuncFormatter(lambda v, _:
    ↪ de(v, 2)))
    ax.yaxis.set_major_formatter(FuncFormatter(lambda v, _:
    ↪ de(v, 2)))
    ax.set_xlabel("IBS0 (Anteil entgegengesetzter
    ↪ Homozygoter)", fontsize=8.8)
    ax.set_ylabel("KINSHIP (KING-robust)", fontsize=8.8)
    style_axis(ax, grid=None)


#
    ↪ ------------------------------------------------------------
    ↪  Haupt ----
def save(fig, base):
    base = Path(base)
    base.parent.mkdir(parents=True, exist_ok=True)
    for ext in ("png", "pdf"):
        fig.savefig(base.with_suffix(f".{ext}"), dpi=300,
    ↪ bbox_inches="tight", pad_inches=0.15)
    plt.close(fig)
    print("geschrieben:", base.with_suffix(".png"))


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--mendel", default="results/mendel.tsv")
    ↪
    ap.add_argument("--king", default="results/king.kin0")
    ap.add_argument("--outdir", default="results/abbildungen
    ↪ ")
    a = ap.parse_args()

    pairs, trios = read_mendel(a.mendel)
    king = read_king(a.king)
    names = sorted({p["a"] for p in pairs} | {p["b"] for p in
    ↪  pairs})
    cut = split_pairs(pairs)
    trio = min(trios, key=lambda t: t["rate"]) if trios else
    ↪ None
    out = Path(a.outdir)

    # Keine Titel im Bild: Beschriftung kommt in die
    ↪ Abbildungsunterschrift
    fig, ax = plt.subplots(figsize=(7.4, 4.6))
    draw_network(ax, names, pairs, trio, cut)
    ax.set_xlim(-2.2, 4.35)
    save(fig, out / "abb1_verwandtschaftsnetz")

    fig, ax = plt.subplots(figsize=(7.0, 3.4), layout="
    ↪ constrained")
    draw_pairs(ax, pairs, cut)
    save(fig, out / "abb2_mendel_paare")

    if trios:
        fig, ax = plt.subplots(figsize=(7.4, 4.4), layout="
    ↪ constrained")
        draw_trios(ax, trios)
        save(fig, out / "abb3_mendel_trios")

    fig, ax = plt.subplots(figsize=(6.6, 4.6), layout="
    ↪ constrained")
    draw_king(ax, king, pairs, cut)
    save(fig, out / "abb4_king")


if __name__ == "__main__":
    main()
```